Data Science for Economists

Lecture 5: Loops in R

Jiaxi Li
University of North Carolina | ECON 370

Table of contents

✱ Slides adapted from Drew Van Kuiken and Alex Marsh's ECON 370 and originally based on materials by Grant McDermott's EC 607 at University of Oregon.

Introduction

Agenda

This class will cover loops in R and their quirks

Historically:

- Loops in R were traditionally considered **slow**.
- The `apply` **family** of functions provided cleaner code and was often recommended over explicit loops.
- However, the `apply` functions are still **loops under the hood**, and not fundamentally faster.
- Best practice was to **vectorize operations** whenever possible for efficiency.

Today:

- **Modern R loops are not inherently slow**:
  - Performance is comparable to other languages when written efficiently.
  - Key improvements come from **proper structure** and **pre-allocation**.
- For **computationally heavy tasks**, **parallelization** can drastically reduce runtime.
- We will focus on `future.apply`, which simplifies parallelization.

Loops

Motivation: Parallel and Dynamic Tasks

Parallel tasks: Each task is independent and similar in nature.

Example1.1: Calculate the mean for 10 random numbers **100 times**.

Example1.2: Calculate the mean for 10 random numbers **until find one mean is above 1**.

Dynamic Tasks: Later task depends on the result of previous task but use similar principle.

Example2.1: Generate the **first 20** Fibonacci sequence, starting from $X_1 = 0$ and $X_2 = 1$: $X_{n+2} = X_{n+1} + X_n$ for $n \geq 1$.

Example2.2: Generate Fibonacci sequence **less than 100**, starting from $X_1 = 0$ and $X_2 = 1$: $X_{n+2} = X_{n+1} + X_n$ for $n \geq 1$.

All these tasks can be done with loops!

1.1 and 2.1 stops with a pre-specified number of times. -- use `for` loop.

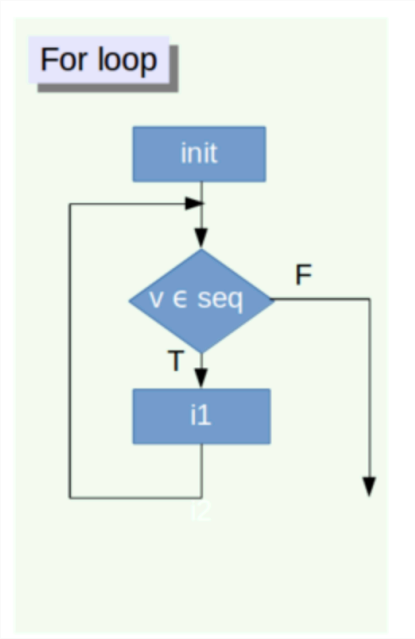
1.2 and 2.2 stops with some logical criteria. -- use `while` loop.

For Loop (Looping Over Index)

At times we would look to do the same task for a fixed number of times that only changes slightly each iteration.

This can be done with a `for` loop.

We can loop over indexes for that vector, list, data.frame, etc.



Our First Loop

Adding numbers from 1 to 10.

```
# Preset the variable
sum_val = 0
# Use for loop to add one number at a time
for(i in 1:10){
  sum_val = sum_val + i
}
sum_val
```

```
## [1] 55
```

```
# Compare with built in R function sum
sum_val == sum(1:10)
```

```
## [1] TRUE
```

For Loop (Looping Over Objects)

We can also loop over objects in a vector, list, data.frame, etc. It works when you just have a list to go over.

For example, with a list of names, I want to add "UNC-" in front of every name.

```
# Here is the vector name list
Names_vec = c("James", "Julia", "John", "Jasmine", "Jack")

for (Name in Names_vec) {
  # Use cat to combine and print
  cat("UNC-", Name, "\n", sep = "")
}
```

```
## UNC-James
## UNC-Julia
## UNC-John
## UNC-Jasmine
## UNC-Jack
```

I personally prefer looping over index since we can keep better track of the progress or save your result!

Here, we might not need a loop. (`cat` can apply to vectors, so...)

Two Approaches: An Example

```
N          = 100          #set number of draws
norm_draws = rnorm(N)     #draw N(0,1) random variables
out1       = rep(NA,N)    #preallocate output 1: MORE ON THIS LATER
out2       = rep(NA,N)    #preallocate output 2: MORE ON THIS LATER
i          = 1            #initialize counter

for(draw in norm_draws){
  out1[i] = draw^2        #square the draw and store it
  i       = i + 1         #advance the counter
} #NOTE THAT A COUNTER IS NEEDED

for(i in 1:N){
  out2[i] = norm_draws[i]^2 #square the ith draw and store it
} #notice no counter needed

all.equal(out1,out2) #test to see if these approaches are the same; they are!
```

```
## [1] TRUE
```

More Examples: Advanced Sums

Suppose we wanted to calculate

$$\sum_{a=1}^{20}\sum_{b=1}^{15}\frac{e^{\sqrt{a}}\log\left(a^5\right)}{5+\cos(a)\sin(b)}$$

```
# Preset the variable
val = 0
# Loop over a
for(A in 1:20){
  # Loop over b
  for(B in 1:15){
    val = val + (exp(sqrt(A))*log(A^5))/(5+cos(A)*sin(B))
  }
}
val
```

```
## [1] 25922.81
```

The loop works, but we can do better here!

- Will return to this at the end of the lecture.

Preallocation

Many times you'll want to use a loop to "fill up" a matrix or vector. Or you want to keep a complete copy of the results for each iteration.

It is best practice to "preallocate" this object to the correct size before filling it up.

There are a few reasons for this, but it ultimately comes down to speed:

- Changing the size of the object inside the loop each iteration makes loops much slower!

I would recommend you to preallocate with `NA`.

Some people preallocate with 0. However, when 0 is in the final result, we do not know whether it is from preallocation or computation.

'NA' is flexible with different data types. And with `NA`, it is easy to track down problems with computation.

Preallocation: Example

```
N          = 10000 # number of iterations
my_vec     = c(NA) # No Preallocation
my_vec_pre = rep(NA,N) # Preallocation

# Use for loop to assign new values to my_vec without preallocation
# Each iteration increases the size of my_vec.
for(i in 1:N){
  my_vec[i] = i^2
}

# We can also use 1:N here, but length() is more flexible in some situations
for(i in 1:length(my_vec_pre)){
  my_vec_pre[i] = i^2
}

all.equal(my_vec, my_vec_pre) # compare results
```

```
## [1] TRUE
```

Both run! But let's look at the speed.

Preallocation: Comparing Speeds

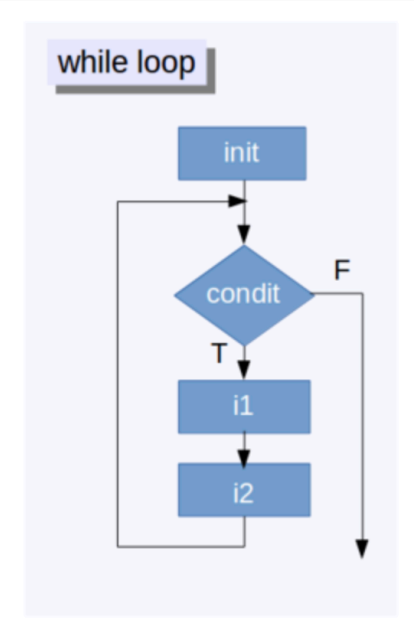
```
# Use microbenchmark to test the speed of no preallocation vs. preallocation
mbm = microbenchmark(
  "no_preal"={
    my_vec     = c(NA)
    for(i in 1:N){
      my_vec[i] = i^2
      }},
  "preal"={
    my_vec_pre = rep(NA,N)
    for(i in 1:length(my_vec_pre)){
      my_vec_pre[i] = i^2
      }},times=1000)
mbm
```

```
## Unit: milliseconds
##       expr    min      lq     mean  median      uq     max neval
##  no_preal 2.7248 2.82270 3.167060 2.88030 2.99895 66.4240  1000
##     preal 1.8740 1.94115 2.095453 1.98255 2.05235  8.2964  1000
```

Bottom line: **Always preallocate** if you are saving the result of each iteration.

While loops

- For loops are not the only types of loops in R!
- Another type is while loops.
- Instead of looping through objects or indexes, we continue to do something ***until*** **a condition is no longer met**.
- This can be really useful for some of the things we will use later on.
- Can be dangerous though: infinite loop!
    - Not so much in RStudio, though.



Our First While Loop

Again, we try to add the number from 1 to 10

```
# Preset the variable
sum_val = 0
# Set a counter
i = 1

# Use while loop, keep adding while n is less or equals 10
while(i <= 10){
  sum_val = sum_val + i
  i   = i + 1 #update the counter
}

# Compare with built in R function sum
sum_val == sum(1:10)
```

```
## [1] TRUE
```

The `while` loop continues to increase n by one and add it to `val` until $n > 10$.

This is really a `for` loop!

Application: An Easy Fixed Point

In math, a fixed point, x^*, of a function f is defined as a value where $f(x^*) = x^*$

- So a fixed point is a point where when we apply the function to it, we get the original value back!

While loops can be used to calculate these when they exist.

The idea is to keep applying the function over and over again until the values are "close enough"

So $x_{n+1} = f(x_n)$. If $|x_{n+1} - f(x_{n+1})|$ is "small," we stop.

If not, $x_{n+2} = f(x_{n+1})$, and compare x_{n+2} and $f(x_{n+2})$.

We will use the function $f(x) = \sqrt{x}$.

$\sqrt{1} = 1$ and $\sqrt{0} = 0$. So these are our candidate fixed points.

However, we will only get to one of them no matter which starting values we start at.

Calculating Fixed Points

```
eps   = .Machine$double.eps #set tolerance
x_n   = 5000                #starting guess

# Keep looping until x_n is sufficiently close to sqrt(x_n)
while(abs(x_n - sqrt(x_n)) > eps){
  x_n = sqrt(x_n) #update guess
}
x_n
```

```
## [1] 1
```

```
x_n   = 0.00001             #starting guess

# Keep looping until x_n is sufficiently close to sqrt(x_n)
while(abs(x_n - sqrt(x_n)) > eps){
  x_n = sqrt(x_n) #update guess
}
x_n
```

```
## [1] 1
```

Finance Application: Stock Returns

If you invest in stock market, you can generate `returns`, which is the amount extra money you make divided by your investment amount.

For demonstration purpose, let's get SP500 index data (with symbol `^GSPC`) from Yahoo Finance. We can use `tidyquant` package to extract the data (`tidyquant` can be used to extract more than just Yahoo Finance data).

Let's suppose that you can buy and sell the SP500 index at the adjusted closing price (you can look into the details of different variables and what is adjusted closing price if you are interested in finance).

SP500 Monthly Price

Let's extract the stock price since 12/29/2017:

```
library(tidyquant)

# Get daily S&P 500 data (or Global Standard & Poor's Composite, symbol: ^GSPC)
SP500_monthly = tq_get("^GSPC",
                       from = "2017-12-29",
                       to = "2025-08-29",
                       get  = "stock.prices") ▷
  # only keep the data at the end of each month
  tq_transmute(select = adjusted,          # use adjusted closing price
               mutate_fun = to.monthly,    # collapse to monthly
               indexAt = "lastof",         # use last day of month for date
               col_rename = "price") ▷     # create new name for the column
  as.data.frame() # change to data frame since tq_get would produce a tibble

head(SP500_monthly)
```

```
##         date   price
## 1 2017-12-31 2673.61
## 2 2018-01-31 2823.81
## 3 2018-02-28 2713.83
## 4 2018-03-31 2640.87
## 5 2018-04-30 2648.05
```

Stock Returns

The stock price is the price of one share. However, in reality, people rarely just hold one share. So it is more useful to look at the stock returns instead of stock prices:

$$r_{t+1} = \frac{P_{t+1} - P_t}{P_t} = \frac{P_{t+1}}{P_t} - 1$$

```
##         date   price
## 1 2017-12-31 2673.61
## 2 2018-01-31 2823.81
```

If we buy one share of S&P500 index at the end of December 2017 and sell it at the end of January 2018, how much do we make? What is the percentage of money increase (return)?

If instead, we buy two shares of S&P500 index at the end of December 2017 and sell them at the end of January 2018, how much do we make? What is the percentage of money increase (return)?

Calculate Monthly Returns in R

Therefore, we can just use stock return to see how my money is growing, regardless of how much stock I buy. Let's try to calculate the monthly returns each month.

```
# Preallocate as a new variable in data.frame, set value as NA
# Try not to set value at 0 if 0 has meanings
SP500_monthly$Return = NA

# Use for loops from the second index to the last
for (i in 2:nrow(SP500_monthly)) {
  SP500_monthly$Return[i] = SP500_monthly$price[i]/SP500_monthly$price[i-1] - 1
}

head(SP500_monthly, n=3)
```

```
##         date   price      Return
## 1 2017-12-31 2673.61          NA
## 2 2018-01-31 2823.81  0.05617870
## 3 2018-02-28 2713.83 -0.03894737
```

Why there is an `NA` at the beginning?

What would happen if we preallocated using 0?

Useful Functions and Controls in Loops

Here's a compact list of useful functions and controls often used in R loops:

- `cat()` – Concatenates and prints text without extra formatting (no quotes, no line breaks unless added manually).
- `message()` / `warning()` - Print message/warning. Can use `suppressMessages()` / `suppressWarnings()` to suppress them.
- `stop()` – Immediately halts the loop (and the whole function/script) with an error.
- `break` – Exits the loop early and skips all remaining iterations. (but still operate things after loop.)
- `next` – Skips the current iteration and jumps to the next one.
- `tryCatch()` – Lets the loop continue even if an error occurs in one iteration, useful for error handling.

Keep track of the progress

Sometimes, the number of total iteration is too large. We want to know how whether the computer freezes or it is still working in progress.

We can simply use `cat` to paste and print the progress. You can use `message` or `warning` as well.

```
# Print the progress every 20th iteration
for (i in 1:100) {
  # Make the system stop for 0.05 second
  Sys.sleep(0.05)
  # Print the progress with cat (You can try print, but I prefer cat here)
  # cat can combine and print the character, but without quotation mark and [1]
  # "\n" means start a new line, you can try what happens without it
  if (i %% 20 == 0) cat("Completed", i, "iterations out of 100\n")
}
```

```
## Completed 20 iterations out of 100
## Completed 40 iterations out of 100
## Completed 60 iterations out of 100
## Completed 80 iterations out of 100
## Completed 100 iterations out of 100
```

Note: Auto print does not work in loops!

Implementing Fail-Safes

When writing while loops, it is often good practice to implement a **fail-safe** so that the while loop doesn't run for forever.

This could be because the code isn't converging as quickly as we'd like or there's an error and the code will never converge because we wrote it wrong.

To implement a fail-safe, we need to create a new variable and use some of the logical properties we talked about last lecture.

Ideas?

Besides $|x_n - f(x_n)| < \varepsilon$ in the while condition, we also want to stop when $n > \bar{N}$ where $\bar{N}$ is some maximum number of iterations we set.

Implementing a Fail-Safe

```
x_n   = 50000  #starting guess
i     = 1      #initialize counter
MaxIt = 100000 # max iteration

while(abs(x_n - sqrt(x_n)) > eps){
  x_n = sqrt(x_n)   #update x_n
  i     = i + 1     #increase counter

  # Stop the code if it takes too many iterations
  if(i >= MaxIt) stop("Did not find fixed point!")
  # Have a message if the loop stops due to condition met
  if(abs(x_n - sqrt(x_n)) <= eps) message(paste("The estimated fix point is approximat
}
```

```
## The estimated fix point is approximately 1 with 55 iterations.
```

Fail-Safes (Fixed Points)

Fixed points don't always exist, and even when they do, we're not always guaranteed to find them via the iterative procedure I described.

That's where these fail safes can come into play.

Consider the function $f(x) = 2x$. f has a fixed point (and only one fixed point) at $x = 0$; however, we are not guaranteed to ever find it via the iterative procedure above.

Therefore, the fail safe needs to be triggered so our loop doesn't go on forever.

Necessary Fail-Safes

```
x_n   = 0.0001 #starting guess
i     = 1      #initialize counter
MaxIt = 1000   #fix max iterations

while(abs(x_n - 2*x_n) > eps){
  x_n = 2*x_n   #update x_n
  i     = i + 1     #increase counter

  # Stop the code if it takes too many iterations
  if(i >= MaxIt) stop("Did not find fixed point!")
  # Have a message if the loop stops due to condition met
  if(abs(x_n - 2*x_n) <= eps) message(paste("The estimated fix point is approximately'
}
```

```
## Error: Did not find fixed point!
```

Skipping some lines

Sometimes, we would like to skip some lines. For example, for $f(x) = 2x$, how about just step in some random direction and only keep the steps that is favorable to us?

```
x_n    = 0.0001 #starting guess
i      = 1      #initialize counter
MaxIt = 1000    #fix max iterations
UpdateStep = 0.000002    #make the UpdateStep relatively small

while(abs(x_n - 2*x_n) > eps){
  x_temp = x_n + UpdateStep*sample(c(-1,1), 1) # have the temporary x that is x updat
  i = i + 1 #increase counter

  # Stop the code if it takes too many iterations
  if(i ⩾ MaxIt) stop("Did not find fixed point!")
  if(abs(x_temp - 2*x_temp) ⩽ eps) message(paste("The estimated fix point is", x_temp

  # skip updating x if it not in favorable direction
  if(abs(x_temp - 2*x_temp) ⩾ abs(x_n - 2*x_n)) next

  x_n = x_temp # update x_n
}
```

```
## The estimated fix point is 8.72443935671929e-20 with 91 iterations.
```

tryCatch

Sometimes, we would have only a few error break the entire loop. It would be nice to finish the loop and then look back.

```
# Use tryCatch to suppress error and print a message instead
# We can save the index and come back later
Warning_ind = NULL
for (i in 1:5) {
  tryCatch({
    if (i == 2) stop("Error at i=2")
    if (i %in% 3:4) warning("Warning at i=3 and i=4")
  }, error = function(e) {
    message("Skipping error at i=", i)
  }, warning = function(e) {
    Warning_ind <<- c(Warning_ind, i) # Append index to Warning_ind
  })
}
```

```
## Skipping error at i=2
```

```
Warning_ind
```

```
## [1] 3 4
```

Repeat Loops

In R, there is a third kind of loop: the repeat loop.

The repeat loop will continue to do something until you manually break it.

These are slightly different than while loops; however, while loops can be used to replicate their behavior quite easily.

I would mostly recommend avoiding repeat loops.

Repeat Loops

```
# Preset value and index
val = 0
n   = 1
# Repeat will keep running until manual break
repeat{
  val = val + n
  n   = n + 1
  if(val > 30) break  # Condition for manual break
}
print(c(val, n))
```

```
## [1] 36  9
```

```
# Preset value and index
val = 0
n   = 1
# Break when val > 30 (i.e. not break when val ⩽ 30)
while(val ⩽ 30){
  val = val + n
  n   = n + 1
}
print(c(val, n))
```

The Apply Family of Functions

The Apply Family

In R, there are a family of functions called the `apply` family.

They can be used to write loops in a much more compact format.

The idea is to have some vector-like object that you would to do something to in a for-loop like manner, and then "apply" some function to each element of the object.

If you'd like to see more about the `apply` family, you can follow the `swirl` tutorial for more.

The Apply Function

The first one we will look at is the `apply` function.

It takes three arguments:

1. an array (matrix, vector, etc.)
2. a "margin" (which dimension to apply over)
3. a function

It takes the array and then applys the function over the dimension that is specified in the margins argument.

Apply: An Example

```
# Create a random matrix with rnorm. Remember to use set.seed.
set.seed(42)
rand_mat = matrix(rnorm(3*2),ncol=3)
rand_mat
```

```
##            [,1]      [,2]       [,3]
## [1,]  1.3709584 0.3631284  0.4042683
## [2,] -0.5646982 0.6328626 -0.1061245
```

```
apply(rand_mat,1,sum)
```

```
## [1]  2.13835518 -0.03796008
```

```
apply(rand_mat,2,sum)
```

```
## [1] 0.8062603 0.9959910 0.2981438
```

- MARGIN = 1, the sum function is applied to each row.
  - So we are summing across columns
- MARGIN = 2, the sum function is applied to each column.
  - So we are summing across rows.

Apply's Connection to Loops

It might not be entirely obvious apply's connection to loops.

When MARGIN = 1, this is what `apply` is doing:

```
# Preallocation with NA
out = rep(NA,nrow(rand_mat))
for(i in 1:nrow(rand_mat)){
  out[i] = sum(rand_mat[i,])
}
out
```

```
## [1]  2.13835518 -0.03796008
```

Likewise, when MARGIN = 2, this is what `apply` is doing:

```
# Preallocation with NA
out = rep(NA,nrow(rand_mat))
for(i in 1:ncol(rand_mat)){
  out[i] = sum(rand_mat[,i])
}
out
```

```
## [1] 0.8062603 0.9959910 0.2981438
```

Beyond Apply

As seen above, apply can simplify loops and results in much cleaner code.

- Though, is it more readable?

While the `apply` function is useful, it has its limitations.

1. It can only be used on array-like objects.
2. It will only return a vector or array.

There are other functions that can be used on a wider class of objects along with return non-arrays.

- `lapply`: returns a list the same length as the object
- `sapply`: returns the "most simple" version of the output of lapply that makes sense.
  - I know, this sounds ambiguous because it is!
- `vapply`: the same as sapply, but an output type must be specified.
  - Generally, safer to use.
- `replicate`: Repeat some operation multiple times
  - Great for simulation or repeated process

x-apply Examples

```r
# Create a list to apply functions
my_list = list(a = 1:10, beta = exp(-3:3), logic = c(TRUE,FALSE,FALSE,TRUE))
my_list
```

```
## $a
##  [1]  1  2  3  4  5  6  7  8  9 10
##
## $beta
## [1]  0.04978707  0.13533528  0.36787944  1.00000000  2.71828183  7.38905610
## [7] 20.08553692
##
## $logic
## [1]  TRUE FALSE FALSE  TRUE
```

x-apply Examples (Cont.)

```
# calculate the mean of each object in the list with lapply
lapply(my_list, mean)
```

```
## $a
## [1] 5.5
##
## $beta
## [1] 4.535125
##
## $logic
## [1] 0.5
```

```
# sapply default returns a vector or matrix
sapply(my_list, mean)
```

```
##        a     beta    logic
## 5.500000 4.535125 0.500000
```

x-apply Examples (Cont.)

```r
# lapply quantile function and `probs = (1:3)/4` is the argument for quantile
lapply(my_list, quantile, probs = (1:3)/4)
```

```
## $a
##  25%  50%  75%
## 3.25 5.50 7.75
##
## $beta
##       25%       50%       75%
## 0.2516074 1.0000000 5.0536690
##
## $logic
## 25% 50% 75%
## 0.0 0.5 1.0
```

```r
# sapply default returns a vector or matrix
sapply(my_list, quantile)
```

```
##          a        beta logic
## 0%    1.00  0.04978707   0.0
## 25%   3.25  0.25160736   0.0
## 50%   5.50  1.00000000   0.5
## 75%   7.75  5.05366896   1.0
```

x-apply Examples (Cont.)

By default, `sapply` will apply functions to columns (across rows) of data.frames. i.e. MARGIN = 2 in the apply function.

Note, there is no MARGIN argument for `sapply`, `lapply`, or `vapply.`

```
data(mtcars)
sapply(mtcars,summary)
```

```
##              mpg    cyl     disp       hp     drat      wt     qsec     vs
## Min.    10.40000 4.0000  71.1000  52.0000 2.760000 1.51300 14.50000 0.0000
## 1st Qu. 15.42500 4.0000 120.8250  96.5000 3.080000 2.58125 16.89250 0.0000
## Median  19.20000 6.0000 196.3000 123.0000 3.695000 3.32500 17.71000 0.0000
## Mean    20.09062 6.1875 230.7219 146.6875 3.596563 3.21725 17.84875 0.4375
## 3rd Qu. 22.80000 8.0000 326.0000 180.0000 3.920000 3.61000 18.90000 1.0000
## Max.    33.90000 8.0000 472.0000 335.0000 4.930000 5.42400 22.90000 1.0000
##              am   gear   carb
## Min.    0.00000 3.0000 1.0000
## 1st Qu. 0.00000 3.0000 2.0000
## Median  0.00000 4.0000 2.0000
## Mean    0.40625 3.6875 2.8125
## 3rd Qu. 1.00000 4.0000 4.0000
## Max.    1.00000 5.0000 8.0000
```

replicate

We can replicate some process, this usually happens in simulation.

```
# Draw 10 random numbers with rnorm. Repeat this 6 times and save into a matrix. Do n
set.seed(42)
Rep_matrix = replicate(4, rnorm(6))
Rep_matrix
```

```
##            [,1]        [,2]       [,3]       [,4]
## [1,]  1.3709584  1.51152200 -1.3888607 -2.4404669
## [2,] -0.5646982 -0.09465904 -0.2787888  1.3201133
## [3,]  0.3631284  2.01842371 -0.1333213 -0.3066386
## [4,]  0.6328626 -0.06271410  0.6359504 -1.7813084
## [5,]  0.4042683  1.30486965 -0.2842529 -0.1719174
## [6,] -0.1061245  2.28664539 -2.6564554  1.2146747
```

```
# Draw 6*4 random numbers with rnorm and then reshape it into a matrix. Do not forget
set.seed(42)
Reshaped_matrix = matrix(rnorm(6*4), ncol = 4)

all.equal(Rep_matrix, Reshaped_matrix)
```

```
## [1] TRUE
```


The reshaped matrix is faster and preferred though. Why?

Vectorization

To Loop or Not To Loop

Vector operations are much faster than loops!

```
mbm = microbenchmark(
  "loop"={
    N   = 10000    #set size of vector
    out = rep(0, N) #preallocate vector
    for(i in 1:N){
      out[i] = i^2  #fill in vector with square of index
      }},
  "vectorized"={
    N   = 10000    #set size of vector
    out = 1:N       #preallocate vector
    out = out^2     #square each index
   },times=1000)
mbm
```

```
## Unit: microseconds
##        expr    min     lq      mean  median     uq    max neval
##        loop 1720.2 1772.3 1881.4015 1798.50 1842.6 6042.3  1000
##  vectorized   14.6   16.9   26.2618   19.35   23.0  161.1  1000
```

Returning To Advanced Sums

Earlier, we wanted to calculate the following sum:

$$\sum_{a=1}^{20}\sum_{b=1}^{15}\frac{e^{\sqrt{a}}\log\left(a^5\right)}{5+\cos(a)\sin(b)}$$

While we used a loop, it was not necessary. If we expand out every combination of a and b, then, we can use vectorized operations.

```
# "Vectorize a matrix". Now, aANDb contains all pairs of a and b
aANDb = expand.grid(A=1:20,B=1:15)
A     = aANDb$A
B     = aANDb$B
# Run the sum on all pairs of a and b
sum((exp(sqrt(A))*log(A^5))/(5+cos(A)*sin(B)))
```

```
## [1] 25922.81
```

Benchmarking These Sums

```
# Test the speed of loop vs. vectorized advanced sum
mbm = microbenchmark(
  "loop"={
    val = 0
    for(A in 1:20){
      for(B in 1:15){
        val = val + (exp(sqrt(A))*log(A^5))/(5+cos(A)*sin(B))}}},
  "vectorized"={
    aANDb = expand.grid(A=1:20,B=1:15)
    A     = aANDb$A
    B     = aANDb$B
    sum((exp(sqrt(A))*log(A^5))/(5+cos(A)*sin(B)))
  },times=1000)
mbm
```

```
## Unit: microseconds
##         expr    min      lq      mean  median      uq     max neval
##         loop 4892.4 5024.45 5385.5191 5107.15 5301.90 14343.8  1000
##  vectorized  131.3  147.60  183.3199  191.80  216.05   359.1  1000
```

```
mean(mbm[mbm$expr=="loop","time"])/mean(mbm[mbm$expr=="vectorized","time"])
```

```
## [1] 29.37771
```

Vectorization with Matrix/Array

We just see that operating on a vector can be much faster than a loop!

This is also true for higher dimensional data type: matrix and array.

This is why linear algebra is important for more advanced data science.

For a data.frame with the same data type across all columns, it is essentially a matrix. Let's look back at the data.frame we used in lecture4.

```
# Create a data.frame with three variables
# Set seed due to randomness
set.seed(2025)
# x is from uniform distribution, y is from normal distribution and z is from chi squ
d = data.frame(x=runif(6),y=rnorm(6),z=rchisq(6,1))
```

Vectorization with Matrix/Array

We can take the column mean one column at a time with loop, but we can also use matrix operating functions (we still call this vectorization, but with matrix)!

```
# We can use apply to demonstrate here since it is basically a loop
apply(d, 2, mean)
```

```
##         x         y         z
## 0.5842611 0.2421279 1.1808079
```

```
# colMeans is a function for matrix
colMeans(d)
```

```
##         x         y         z
## 0.5842611 0.2421279 1.1808079
```

Compare Speeds

```r
# Create a large numeric matrix
set.seed(123)
big_mat = matrix(rnorm(1e7), nrow = 1000, ncol = 10000)  # 1000 x 10000

benchmark_result = microbenchmark(
  base_colMeans = colMeans(big_mat),    # Optimized base R function
  apply_mean = apply(big_mat, 2, mean),# General-purpose apply
  times = 10                            # Repeat each test 10 times
)

# Print detailed timing results
print(benchmark_result)
```

```
## Unit: milliseconds
##           expr      min       lq      mean   median       uq      max neval
##  base_colMeans   9.3672   9.5098   9.87787   9.5687   9.7685  12.3943    10
##     apply_mean 139.0726 146.0562 148.04820 147.4354 148.7114 159.5839    10
```

Note: You can use `replicate` to create the big matrix, but it would be slower.

Note: If you know linear algebra, you can also compare speed of matrix multiplication (`%*%`) and corresponding loops.

Parallelization with future.apply

Parallelization

Parallelization in R means splitting work across multiple CPU cores to make things faster. By default, R only uses one core, even if you have many.

There are many ways to conduct parallel computation. We will work with the easy parallel apply with `future.apply`.

This package makes it super easy—just replace `x-apply()` with `future_x-apply()`.

```
library(future.apply)

# Detect the number of cores on your laptop
parallel::detectCores()  # from base R's parallel package
```

```
## [1] 20
```

```
plan(multisession, workers = 4)  # use 4 cores

my_list = list(a = 1:10, beta = exp(-3:3), logic = c(TRUE,FALSE,FALSE,TRUE))

# future_lapply(my_list, mean)

all.equal(unlist(lapply(my_list, mean)), unlist(future_lapply(my_list, mean)))
```



```
## [1] TRUE
```

Best Practice with future.apply

Don’t oversubscribe: If you have 8 cores, don’t always use all 8—leave some for your system.

Large tasks benefit most: Parallel overhead is costly, so use it for heavier tasks (like simulations, bootstrapping, large data processing).

Same as apply family, `future.apply` only work with parallel tasks.

When generating random numbers, make sure to use `future.seed = xxx` to avoid unintended overlap (duplication) and non-reproducibility.

Benchmark Different Approaches: I have included the benchmarking code in the script. I will just show the result:

```
## Unit: milliseconds
##         expr       min        lq      mean    median        uq       max neval
##   for_loop() 1288.1779 1293.9699 1314.7966 1299.7620 1328.1059 1356.4498     3
##  apply_seq() 1331.6848 1332.9895 1356.9916 1334.2942 1369.6450 1404.9958     3
##  apply_par()  516.6633  529.5994  535.2828  542.5355  544.5925  546.6495     3
```

Dynamic Tasks

When Should We Use Loops?

Sometimes, especially for dynamic tasks, the use of a loop is hard to avoid. Here are some examples:

1. Calculations depend on previous calculations.
2. The size of an "inner loop" changes based on the values of the "outer loop."
3. Too difficult to do the "prep-work" mentally for the vectorized operations.

Calculations That Depend on Others

An $AR(1)$ Time Series is a perfect example of an economic application where a loop is absolutely necessary.

An $AR(1)$ model says that today's value of y, called y_t, depends on yesterday's, y_{t-1}, scaled by some value ρ, plus some constant δ, plus some error term ε_t.

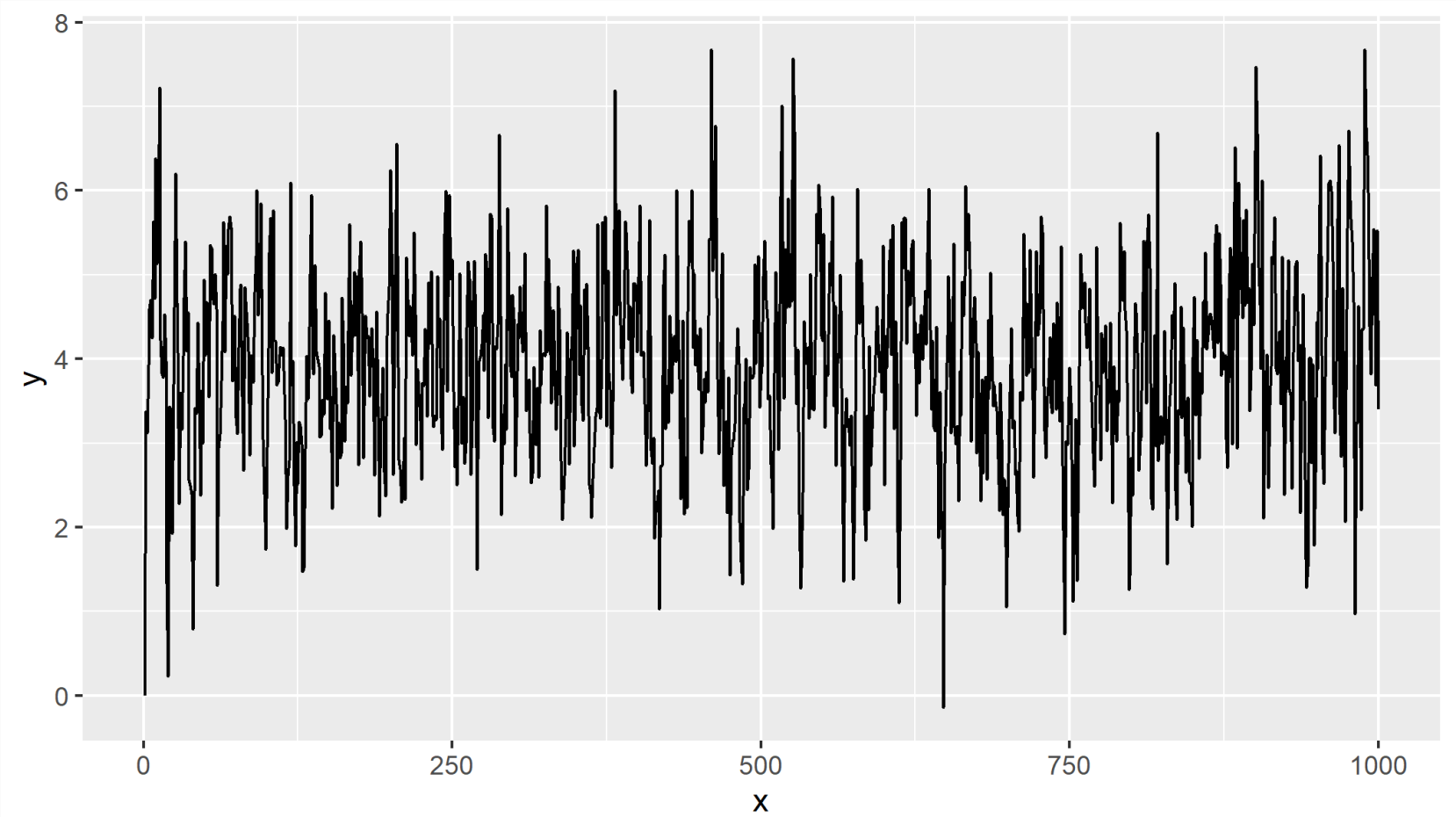
In math, that is

$$y_t = \delta + \rho y_{t-1} + \varepsilon_t$$

```
# Set parameters, number of period N, set seed for randomness
rho       = 0.5
delta     = 2
N         = 1000
set.seed(42)

# Preallocate and set first value at 0
AR1_ts    = rep(NA,N)
AR1_ts[1] = 0
# Use for loop updating remaining values
for(i in 2:N){
  AR1_ts[i] = delta + rho*AR1_ts[i-1] + rnorm(1)
}
```

AR(1) Plot



Temporarily Saved Values

Sometimes, we only want to save the **final result** of a dynamic task instead of saving all results. We already see them in the fixed point example with *Skipping some lines*.

If we will need more than one previous value for calculation later, it is common to temporarily save a variable such as `Val_pre`. If we use up to `k`th previous value in dynamic task, we might want to have a vector of length `k`.

For example, for AR(3) process,

$$y_t = \delta + \rho_1 y_{t-1} + \rho_2 y_{t-2} + \rho_3 y_{t-3} + \varepsilon_t$$

We would need three `Val_pre`. Let's only keep the final value of the AR(3) process:

AR(3) at N=1000

```r
# Set parameters, number of period N, set seed for randomness
rho1      = 0.3
rho2      = 0.2
rho3      = 0.1
delta     = 2
N         = 1000
set.seed(42)

# set pre_value at 0 for the three most recent history (t-1, t-2, t-3)
AR1_ts_pre = rep(0,3)
# Use for loop updating N times
for(i in 1:N){
  AR1_ts = delta + rho1*AR1_ts_pre[1] + rho2*AR1_ts_pre[2] + rho3*AR1_ts_pre[3] + rnor
  # Then, we need to update the most recent 3 values
  AR1_ts_pre = c(AR1_ts, AR1_ts_pre[-length(AR1_ts_pre)])
}

AR1_ts
```

```
## [1] 4.8121
```

Inner Loop Dependency

Sometimes when loops are nested (like our advanced sums), the inner loops will depend on values of the outer loop.

In this case, loops cannot be entirely avoided.

- Though, they can be minimized.

Consider a slight modification of the advanced sum we saw earlier

$$\sum_{a=1}^{20}\sum_{b=1}^{a}\frac{e^{\sqrt{a}}\log\left(a^5\right)}{5+\cos(a)\sin(b)}$$

Instead of looping b from 1 to 15, now the max value of b depends on the current value of a.

In this case, a loop cannot is hard to avoid. You need to think hard to vectorize.

Dependent Loops

```
# Preset val
val = 0
# loop over a
for(a in 1:20){
  # loop over b, notice the inner loop is from 1 to a (depends on the value of a)
  for(b in 1:a){
    val = val + (exp(sqrt(a))*log(a^5))/(5+cos(a)*sin(b))
  }
}
val
```

```
## [1] 27100
```

If the speed of your code matters, it is worth spending the time to vectorize!

How should we vectorize this?

Vectorized Dependent Loops

This is a relatively simple case since b cannot be bigger than a. We can obtain the grid like before, and then remove any b>a.

```
# "Vectorize a matrix". Now, aANDb contains all pairs of a and b
aANDb = expand.grid(A=1:20,B=1:15)
# Use logical index to select b⩽a
aANDb_depend = aANDb[aANDb$B⩽aANDb$A, ]
A     = aANDb_depend$A
B     = aANDb_depend$B
# Run the sum on all pairs of a and b
sum((exp(sqrt(A))*log(A^5))/(5+cos(A)*sin(B)))
```

```
## [1] 23597.56
```

In Conclusion

Loops:

- Very valuable to understand conceptually.
- Pre-allocation is very important to ensure speed.
- Work for parallel and dynamic tasks.

Apply family :

- Condense loops into more compact syntax. However, they are still loops at heart.

For Parallel Tasks (not Dynamic):

- For large amount of parallel computations, use `future.apply` to do parallel computation.
- **Vectorize if you can!**

For further experience, use `swirl` for loops and the apply functions.

For further reading, please see this link. It was very helpful when making these slides.

Next lecture(s): Miscellaneous